

Updated Edge Functions

Here they are:

EDGE FUNCTION 1 — BILLING MANAGER

```
// supabase/functions/billing-manager/index.ts

import { serve } from "<https://deno.land/std@0.168.0/http/server.ts>"
import { createClient } from "<https://esm.sh/@supabase/supabase-js@2>"

const supabase = createClient(
  Deno.env.get("SUPABASE_URL")!,
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
)

const ESTIMATED_XP_COST: Record<string, number> = {
  mentor_message: 5,
  assistant_message: 2,
  roadmap_assistant_message: 2,
  journal_assistant_message: 1,
  image_upload: 3,
  roadmap_generation: 10,
  roadmap_recalibration: 8,
  weekly_review: 6,
  daily_review: 3,
  monthly_breakdown: 10,
  voice_session_free: 40,
  voice_session_paid: 20,
  deep_research: 18,
  context_summarization: 3, // Fix 2 – summarization now billed
```

```

    summarization: 0,
    memory_tagging: 0,
    journal_classification: 0,
    intent_detection: 0,
    chat_pattern_analysis: 0,
  }

const DAILY_TOKEN_CEILING: Record<string, number> = {
  free: 50000,
  builder: 150000,
  operator: 400000,
  founder: 1000000,
}

// Fix 1 – Atomic reservation via Supabase RPC
// Single SQL operation – check and reserve with no race wind
ow
// SQL equivalent:
// UPDATE user_xp
// SET reserved_xp = reserved_xp + p_amount
// WHERE user_id = p_user_id
// AND (earned_xp + purchased_xp + rollover_xp - reserved_xp)
// >= p_amount
// Returns true if row was modified, false if insufficient XP
async function atomicReserveXp(
  userId: string,
  amount: number
): Promise<boolean> {
  const { data, error } = await supabase.rpc("atomic_reserve_
xp", {
    p_user_id: userId,
    p_amount: amount,
  })
  if (error) return false
  return data === true
}

```

```

// Fix 3 – Cancel reservation with retry and dead letter
async function cancelReservationWithRetry(
  userId: string,
  reservationId: string,
  amount: number,
  feature: string
): Promise<void> {
  const maxAttempts = 3
  let backoffMs = 200

  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
    try {
      const { error } = await supabase.rpc("release_xp_reservation", {
        p_user_id: userId,
        p_amount: amount,
      })

      if (!error) {
        // Mark reservation as cancelled
        await supabase
          .from("xp_reservations")
          .update({ status: "cancelled" })
          .eq("id", reservationId)
        return
      }

      throw new Error(error.message)
    } catch (ex) {
      console.warn(`cancelReservation attempt ${attempt}/${maxAttempts} failed for user ${userId}:`, ex)

      if (attempt === maxAttempts) {
        // All retries exhausted – write to dead letter table
        await supabase.from("dead_letter_reservations").insert

```

```

t({
    user_id: userId,
    amount,
    reason: "cancel_failed",
    status: "pending",
    created_at: new Date().toISOString(),
  })
  console.error(`cancelReservation permanently failed f
or user ${userId} - written to dead_letter_reservations`)
} else {
  await new Promise((resolve) => setTimeout(resolve, ba
ckoffMs))
  backoffMs *= 2
}
}
}
}
}

```

```

// Deduction order: rollover first, purchased second, earned
third
// Fix: purchased XP pool now also clamped to zero
async function deductFromPools(
  userId: string,
  amount: number
): Promise<boolean> {
  const { data: xp, error } = await supabase
    .from("user_xp")
    .select("*")
    .eq("user_id", userId)
    .single()

  if (error || !xp) return false

  // Cap actualXp at 3x reservedAmount as safety rail
  // Prevents runaway sessions from generating enormous deduc
tions

```

```

const cappedAmount = Math.min(amount, (xp.reserved_xp ?? 0)
* 3 || amount)

let remaining = cappedAmount
let rolloverXp = xp.rollover_xp
let purchasedXp = xp.purchased_xp
let earnedXp = xp.earned_xp

// Rollover first
if (rolloverXp > 0) {
  const fromRollover = Math.min(rolloverXp, remaining)
  rolloverXp -= fromRollover
  remaining -= fromRollover
}

// Purchased second – clamped to zero
if (remaining > 0 && purchasedXp > 0) {
  const fromPurchased = Math.min(purchasedXp, remaining)
  purchasedXp = Math.max(0, purchasedXp - fromPurchased)
  remaining -= fromPurchased
}

// Earned third – clamped to zero
if (remaining > 0 && earnedXp > 0) {
  const fromEarned = Math.min(earnedXp, remaining)
  earnedXp = Math.max(0, earnedXp - fromEarned)
  remaining -= fromEarned
}

if (remaining > 0) return false

const { error: updateError } = await supabase
  .from("user_xp")
  .update({
    rollover_xp: rolloverXp,
    purchased_xp: purchasedXp,

```

```

        earned_xp: earnedXp,
        last_updated: new Date().toISOString(),
    })
    .eq("user_id", userId)

    return !updateError
}

async function writeTransactionLog(
    userId: string,
    transactionType: string,
    amount: number,
    feature: string,
    tokensConsumed: number,
    modelUsed: string
) {
    await supabase.from("xp_transactions").insert({
        user_id: userId,
        transaction_type: transactionType,
        amount,
        feature,
        tokens_consumed: tokensConsumed,
        model_used: modelUsed,
        created_at: new Date().toISOString(),
    })
}

serve(async (req) => {
    const {
        action,
        userId,
        feature,
        totalTokens,
        modelUsed,
        reservedAmount,
        reservationId,
    } = req.query

```

```

} = await req.json()

// -----
// PREFLIGHT – atomic check and reserve
// Fix 1: single atomic SQL operation
// No race condition window
// -----
if (action === "preflight") {
  const estimate = ESTIMATED_XP_COST[feature] ?? 2

  const { data: user } = await supabase
    .from("users")
    .select("tier")
    .eq("id", userId)
    .single()

  const { data: xp } = await supabase
    .from("user_xp")
    .select("daily_tokens_used, reserved_xp")
    .eq("user_id", userId)
    .single()

  if (!user || !xp) {
    return new Response(
      JSON.stringify({ allowed: false, reason: "user_not_found" }),
      { status: 404 }
    )
  }

  // Check daily ceiling first – cheap read, no lock needed
  const ceiling = DAILY_TOKEN_CEILING[user.tier] ?? 50000
  if (xp.daily_tokens_used >= ceiling) {
    return new Response(
      JSON.stringify({ allowed: false, reason: "daily_limit_reached" }),

```

```

        { status: 200 }
      )
    }

    // Fix 1: Atomic reserve – check and increment in one SQL
statement
    const reserved = await atomicReserveXp(userId, estimate)

    if (!reserved) {
      return new Response(
        JSON.stringify({ allowed: false, reason: "insufficient
t_xp" }),
        { status: 200 }
      )
    }

    // Write reservation record with TTL for sweeper visibili
ty
    const { data: reservationRecord } = await supabase
      .from("xp_reservations")
      .insert({
        user_id: userId,
        amount: estimate,
        feature,
        status: "active",
        created_at: new Date().toISOString(),
        expires_at: new Date(Date.now() + 15 * 60 * 1000).toI
SOString(),
      })
      .select("id")
      .single()

    await writeTransactionLog(
      userId, "reserve", estimate, feature, 0, "preflight"
    )

```



```

return new Response(
  JSON.stringify({
    allowed: true,
    reservedAmount: estimate,
    reservationId: reservationRecord?.id,
  }),
  { status: 200 }
)
}

// -----
// FINALIZE – true-up actual XP cost
// -----
if (action === "finalize") {
  const actualXp = Math.ceil(totalTokens / 2000)

  // Release reservation atomically
  await supabase.rpc("release_xp_reservation", {
    p_user_id: userId,
    p_amount: reservedAmount,
  })

  // Mark reservation record as completed
  if (reservationId) {
    await supabase
      .from("xp_reservations")
      .update({ status: "completed" })
      .eq("id", reservationId)
  }

  // Deduct actual cost from pools in correct order
  const deducted = await deductFromPools(userId, actualXp)

  if (!deducted) {
    await writeTransactionLog(
      userId, "deduction_failed", actualXp, feature, totalT

```

```

tokens, modelUsed
    )
    return new Response(
        JSON.stringify({ success: false, reason: "deduction_f
ailed" }),
        { status: 200 }
    )
}

// Update daily and monthly token counters
const { data: currentXp } = await supabase
    .from("user_xp")
    .select("daily_tokens_used, monthly_tokens_used")
    .eq("user_id", userId)
    .single()

if (currentXp) {
    await supabase
        .from("user_xp")
        .update({
            daily_tokens_used: currentXp.daily_tokens_used + to
totalTokens,
            monthly_tokens_used: currentXp.monthly_tokens_used
+ totalTokens,
            last_updated: new Date().toISOString(),
        })
        .eq("user_id", userId)
    }

    await writeTransactionLog(
        userId, "deduct", actualXp, feature, totalTokens, model
Used
    )

    return new Response(
        JSON.stringify({ success: true, xpDeducted: actualXp

```

```

    })),
    { status: 200 }
  )
}

// -----
// CANCEL – Fix 3: retry with dead letter
// -----
if (action === "cancel") {
  await cancelReservationWithRetry(
    userId,
    reservationId,
    reservedAmount,
    feature
  )

  await writeTransactionLog(
    userId, "reservation_cancelled", reservedAmount, feature, 0, "cancelled"
  )

  return new Response(
    JSON.stringify({ success: true }),
    { status: 200 }
  )
}

// -----
// EARN – task completion XP reward
// -----
if (action === "earn") {
  const { data: xp } = await supabase
    .from("user_xp")
    .select("xp_multiplier, earned_xp")
    .eq("user_id", userId)
    .single()

```

```

    if (!xp) {
      return new Response(
        JSON.stringify({ success: false, reason: "user_not_found" }),
        { status: 404 }
      )
    }

    const baseReward = ESTIMATED_XP_COST[feature] ?? 2
    const multipliedAmount = Math.floor(baseReward * xp.xp_multiplier)

    await supabase
      .from("user_xp")
      .update({
        earned_xp: xp.earned_xp + multipliedAmount,
        last_updated: new Date().toISOString(),
      })
      .eq("user_id", userId)

    await writeTransactionLog(
      userId, "earn", multipliedAmount, feature, 0, "task_completion"
    )

    return new Response(
      JSON.stringify({ success: true, xpEarned: multipliedAmount }),
      { status: 200 }
    )
  }
}

// -----
// SWEEP – runs on schedule every 5 minutes
// Releases stuck reservations older than 15 min

```

```

// Retries dead letter entries
// -----
if (action === "sweep") {
    const cutoff = new Date(Date.now() - 15 * 60 * 1000).toISOString()

    // Release expired active reservations
    const { data: expired } = await supabase
        .from("xp_reservations")
        .select("*")
        .eq("status", "active")
        .lt("expires_at", cutoff)

    if (expired) {
        for (const reservation of expired) {
            try {
                await supabase.rpc("release_xp_reservation", {
                    p_user_id: reservation.user_id,
                    p_amount: reservation.amount,
                })
                await supabase
                    .from("xp_reservations")
                    .update({ status: "swept" })
                    .eq("id", reservation.id)
                console.info(`Swept expired reservation ${reservation.id} for user ${reservation.user_id}`)
            } catch (ex) {
                console.error(`Sweeper failed for reservation ${reservation.id}:`, ex)
            }
        }
    }

    // Retry dead letter entries
    const { data: deadLetters } = await supabase
        .from("dead_letter_reservations")

```

```

        .select("*")
        .eq("status", "pending")

    if (deadLetters) {
        for (const dl of deadLetters) {
            try {
                await supabase.rpc("release_xp_reservation", {
                    p_user_id: dl.user_id,
                    p_amount: dl.amount,
                })
                await supabase
                    .from("dead_letter_reservations")
                    .update({
                        status: "resolved",
                        resolved_at: new Date().toISOString(),
                    })
                    .eq("id", dl.id)
                console.info(`Dead letter ${dl.id} resolved for use
r ${dl.user_id}`)
            } catch (ex) {
                console.error(`Dead letter retry failed for ${dl.i
d}:`, ex)
            }
        }
    }

    return new Response(
        JSON.stringify({ success: true }),
        { status: 200 }
    )
}

return new Response(
    JSON.stringify({ error: "unknown_action" }),
    { status: 400 }
)

```

```
)  
})
```

EDGE FUNCTION 2 — CONTEXT ENGINE

```
// supabase/functions/context-engine/index.ts  
  
import { serve } from "<https://deno.land/std@0.168.0/http/server.ts>"  
import { createClient } from "<https://esm.sh/@supabase/supabase-js@2>"  
  
const supabase = createClient(  
  Deno.env.get("SUPABASE_URL")!,  
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!  
)  
  
const LIVE_WINDOW_SIZE = 6  
const TRIGGER_THRESHOLD = 10  
  
async function getLiveWindow(sessionId: string) {  
  const { data } = await supabase  
    .from("memory_live_window")  
    .select("role, content, message_index")  
    .eq("session_id", sessionId)  
    .order("message_index", { ascending: true })  
  return data ?? []  
}  
  
async function getSessionSummary(sessionId: string) {  
  const { data } = await supabase  
    .from("memory_session_summary")  
    .select("*")  
    .eq("session_id", sessionId)  
    .single()  
}
```

```

    return data
  }

  async function getPersistentMemory(userId: string) {
    const { data } = await supabase
      .from("memory_persistent")
      .select("*")
      .eq("user_id", userId)
      .single()
    return data
  }

  async function getRelevantEvents(userId: string) {
    const { data } = await supabase
      .from("memory_events")
      .select("event_type, severity, summary, impact, related_t
ags, created_at")
      .eq("user_id", userId)
      .eq("is_resolved", false)
      .gte("severity", 0.6)
      .order("created_at", { ascending: false })
      .limit(3)
    return data ?? []
  }

  async function getCachedAnalysis(userId: string) {
    const { data } = await supabase
      .from("memory_cached_analysis")
      .select("*")
      .eq("user_id", userId)
      .is("invalidated_at", null)
      .single()
    return data
  }

  async function appendToLiveWindow(

```



```

    userId: string,
    sessionId: string,
    role: string,
    content: string,
    messageIndex: number
  ) {
    await supabase.from("memory_live_window").insert({
      user_id: userId,
      session_id: sessionId,
      role,
      content,
      message_index: messageIndex,
      created_at: new Date().toISOString(),
    })
  }

// Calls DeepSeek V4 Flash – wired to environment variables
// Fix: no longer returns empty string placeholder
async function callDeepSeekFlash(
  prompt: string,
  maxTokens: number = 800
): Promise<string> {
  const response = await fetch(
    `${Deno.env.get("DEEPSEEK_BASE_URL")}/chat/completions`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${Deno.env.get("DEEPSEEK_API_KEY")}`,
      },
      body: JSON.stringify({
        model: Deno.env.get("DEEPSEEK_MODEL"),
        max_tokens: maxTokens,
        messages: [{ role: "user", content: prompt }],
      }),
    },
  )
}

```

```

    }
  )
  const data = await response.json()
  return data.choices?.[0]?.message?.content ?? ""
}

// Fix 2: Summarization now routed through billing manager
// If user cannot afford summarization, degrade gracefully
// to raw truncated window rather than blocking entirely
async function triggerSummarization(
  userId: string,
  sessionId: string,
  liveWindow: any[],
  existingSummary: any
) {
  const toArchive = liveWindow.slice(0, liveWindow.length - LIVE_WINDOW_SIZE)

  if (toArchive.length === 0) return existingSummary

  // Fix 2: Run preflight for summarization cost
  const preflightResponse = await fetch(
    `${Deno.env.get("SUPABASE_URL")}/functions/v1/billing-manager`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")}`,
      },
      body: JSON.stringify({
        action: "preflight",
        userId,
        feature: "context_summarization",
      })),

```

```

    }
  )

  const preflight = await preflightResponse.json()

  if (!preflight.allowed) {
    // Cannot afford summarization – degrade gracefully
    // Return raw truncated live window instead of blocking u
    ser
    console.warn(`User ${userId} cannot afford summarization
    – falling back to raw window`)
    return existingSummary
  }

  const reservedAmount = preflight.reservedAmount
  const reservationId = preflight.reservationId

  const formattedMessages = toArchive
    .map((m: any) => `${m.role}: ${m.content}`)
    .join("\n")

  const previousSummary = existingSummary
    ? `Previous summary: ${JSON.stringify(existingSummary)}
    \n\n`
    : ""

  const summarizationPrompt = `${previousSummary}Summarize th
  e following conversation. Extract ONLY: goals, decisions, com
  mitments, failures, bottlenecks, progress updates, emotional
  indicators, behavioral patterns, strategic insights. Remove a
  ll filler, greetings, repetition. Return structured JSON only
  with keys: strategic_state, active_goals, behavioral_pattern
  s, roadmap_progress, recent_context, active_bottlenecks, acti
  ve_leverage_points, emotional_state. No explanation, no pream
  ble, no markdown.\n\n${formattedMessages}`

```

```

let summaryResponse: string
let totalTokens = 0

try {
  summaryResponse = await callDeepSeekFlash(summarizationPr
ompt)
  totalTokens = Math.ceil(summarizationPrompt.length / 4) +
Math.ceil(summaryResponse.length / 4)
} catch (ex) {
  // Model call failed – cancel reservation and fall back
  await fetch(
    `${Deno.env.get("SUPABASE_URL")}/functions/v1/billing-m
anager`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${Deno.env.get("SUPABASE_SER
VICE_ROLE_KEY")}`,
      },
      body: JSON.stringify({
        action: "cancel",
        userId,
        reservedAmount,
        reservationId,
        feature: "context_summarization",
      }),
    }
  )
  return existingSummary
}

let parsedSummary
try {
  const clean = summaryResponse.replace(/```json|```/g,
"").trim()

```

```

    parsedSummary = JSON.parse(clean)
  } catch {
    // Retry once with stricter instruction
    const retryResponse = await callDeepSeekFlash(
      summarizationPrompt +
        "\\n\\nYour previous response was not valid JSON. Return only the JSON object with no other content."
    )
    try {
      const clean = retryResponse.replace(/```json|```/g,
        "").trim()
      parsedSummary = JSON.parse(clean)
    } catch {
      // Both attempts failed – cancel reservation, fall back
      await fetch(
        `${Deno.env.get("SUPABASE_URL")}/functions/v1/billing-manager`,
        {
          method: "POST",
          headers: {
            "Content-Type": "application/json",
            Authorization: `Bearer ${Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")}` ,
          },
          body: JSON.stringify({
            action: "cancel",
            userId,
            reservedAmount,
            reservationId,
            feature: "context_summarization",
          }),
        }
      )
      return existingSummary
    }
  }
}

```

```

// Finalize billing for successful summarization
await fetch(
  `${Deno.env.get("SUPABASE_URL")}/functions/v1/billing-man
ager`,
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${Deno.env.get("SUPABASE_SERVI
CE_ROLE_KEY")}` ,
    },
    body: JSON.stringify({
      action: "finalize",
      userId,
      feature: "context_summarization",
      totalTokens,
      modelUsed: "DEEPSEEK",
      reservedAmount,
      reservationId,
    }),
  }
)

// Write to session summary
const { data: existing } = await supabase
  .from("memory_session_summary")
  .select("id")
  .eq("session_id", sessionId)
  .single()

if (existing) {
  await supabase
    .from("memory_session_summary")
    .update({
      ...parsedSummary,

```

```

        updated_at: new Date().toISOString(),
    })
    .eq("session_id", sessionId)
} else {
    await supabase.from("memory_session_summary").insert({
        user_id: userId,
        session_id: sessionId,
        ...parsedSummary,
        token_count: totalTokens,
        created_at: new Date().toISOString(),
        updated_at: new Date().toISOString(),
    })
}

return parsedSummary
}

function buildFinalPrompt(
    persistentMemory: any,
    sessionSummary: any,
    events: any[],
    cachedAnalysis: any,
    liveWindow: any[],
    userMessage: string,
    assertivenessLevel: number
): string {
    // Build each section as a string – no String.format interp
    // Fix: avoids brittle format specifier corruption
    const systemPrompt = [
        "You are Opus, an elite AI mentor on the Opus Devia platf",
        "orm.",
        "You are strategic, direct, analytical, and deeply person",
        "alized.",
        "You operate exclusively within the data provided in this",
        "prompt.",
    ]

```

```

    "Never fabricate information about the user not present i
n the provided data.",
    "If data is insufficient, say so explicitly, ask one targ
eted clarifying question,",
    "then give a provisional assessment clearly labelled as b
ased on available data only.",
    `Assertiveness level: ${assertivenessLevel} of 5.` ,
    "Safety floor – non-negotiable regardless of assertivenes
s level:",
    "Never shame the user. Never mock their ideas. Never leav
e them without a forward path.",
    "Challenge ideas and decisions, never the person.",
  ].join(" ")

```

```

const profileSection = persistentMemory
  ? [
    "USER PROFILE:",
    `Stage: ${persistentMemory.user_stage ?? "unknown"}`,
    `Long term goals: ${JSON.stringify(persistentMemory.l
ong_term_goals ?? [])}`,
    `Recurring failures: ${JSON.stringify(persistentMemor
y.recurring_failures ?? [])}`,
    `Strengths: ${JSON.stringify(persistentMemory.strengt
hs ?? [])}`,
    `Weaknesses: ${JSON.stringify(persistentMemory.weakne
sses ?? [])}`,
    `Consistency score: ${persistentMemory.consistency_sc
ore ?? 0}`,
    `Discipline trend: ${persistentMemory.discipline_tren
d ?? "unknown"}`,
    `Archetype: ${persistentMemory.archetype ?? "unknow
n"}`,
    ].join("\n")
    : "USER PROFILE: New user. No history yet."

```

```

const summarySection = sessionSummary

```



```

    ? [
      "SESSION CONTEXT:",
      `Strategic state: ${sessionSummary.strategic_state ??
""}` ,
      `Active goals: ${JSON.stringify(sessionSummary.active
_goals ?? [])}` ,
      `Current bottlenecks: ${JSON.stringify(sessionSummar
y.active_bottlenecks ?? [])}` ,
      `Emotional state: ${sessionSummary.emotional_state ??
"unknown"}` ,
      ].join("\n")
    : "SESSION CONTEXT: Session just started."

const eventsSection =
  events.length > 0
    ? "RECENT EVENTS:\n" +
      events
        .map((e: any) => `- ${e.event_type} (severity ${e.s
everity}): ${e.summary}`)
        .join("\n")
    : ""

const analysisSection = cachedAnalysis
    ? [
      "STRATEGIC ANALYSIS:",
      `Key leverage points: ${JSON.stringify(cachedAnalysi
s.leverage_points ?? {})}` ,
      `Execution profile: ${JSON.stringify(cachedAnalysis.e
xecution_profile ?? {})}` ,
      ].join("\n")
    : ""

const dialogueSection =
  "RECENT DIALOGUE:\n" +
  liveWindow.map((m: any) => `${m.role.toUpperCase()}: ${m.
content}`) .join("\n")

```

```

const currentInput = `CURRENT INPUT: ${userMessage}`

return [
  `SYSTEM: ${systemPrompt}`,
  profileSection,
  summarySection,
  eventsSection,
  analysisSection,
  dialogueSection,
  currentInput,
]
  .filter(Boolean)
  .join("\n\n")
}

// Fix 4: persistMemory with full error handling and retry queue
async function wrapUpSession(userId: string, sessionId: string) {
  const liveWindow = await getLiveWindow(sessionId)
  const sessionSummary = await getSessionSummary(sessionId)

  const combinedContext = [
    sessionSummary ? JSON.stringify(sessionSummary) : "",
    liveWindow.map((m: any) => `${m.role}: ${m.content}`).join("\n"),
  ]
    .filter(Boolean)
    .join("\n")

  const extractionPrompt = [
    "Based on the following conversation extract key facts for long term memory.",
    "Return JSON only with keys: user_stage, long_term_goals, recurring_failures,",
  ]

```

```

    "strengths, weaknesses, execution_patterns, mentor_observ
ations,",
    "consistency_score (0-1 float), discipline_trend (improvi
ng/stable/declining)",
    "last_major_event.",
    "No explanation, no preamble, no markdown.",
    "",
    combinedContext,
  ].join("\\n")

let extractedFacts: string
try {
  extractedFacts = await callDeepSeekFlash(extractionPromp
t)
} catch (ex) {
  // Fix 4: AI extraction failed – queue for retry
  console.error(`extractKeyFacts AI call failed for user
${userId}:`, ex)
  await supabase.from("memory_retry_queue").insert({
    user_id: userId,
    session_id: sessionId,
    combined_context: combinedContext.substring(0, 5000),
    reason: "extraction_failed",
    created_at: new Date().toISOString(),
  })
  return
}

try {
  const clean = extractedFacts.replace(/``json|``/g, "").
trim()
  const facts = JSON.parse(clean)

  // Fix 4: Proper upsert – handles both insert and update
cases
  const { data: existing } = await supabase

```

```

    .from("memory_persistent")
    .select("id")
    .eq("user_id", userId)
    .single()

    if (existing) {
        const { error: updateError } = await supabase
            .from("memory_persistent")
            .update({ ...facts, updated_at: new Date().toISOString() })
            .eq("user_id", userId)

        if (updateError) {
            throw new Error(updateError.message)
        }
    } else {
        const { error: insertError } = await supabase
            .from("memory_persistent")
            .insert({
                user_id: userId,
                ...facts,
                updated_at: new Date().toISOString(),
            })

        if (insertError) {
            throw new Error(insertError.message)
        }
    }

    // Archive session
    await supabase.from("memory_session_archive").insert({
        user_id: userId,
        session_id: sessionId,
        session_summary: combinedContext.substring(0, 2000),
        major_events: [],
        strategic_updates: facts,
    })

```

```

        behavioral_changes: {},
        token_cost: Math.ceil(combinedContext.length / 4),
        archived_at: new Date().toISOString(),
    })

    // Mark session completed
    await supabase
        .from("sessions")
        .update({
            ended_at: new Date().toISOString(),
            status: "completed",
        })
        .eq("id", sessionId)

    // Clear live window
    await supabase
        .from("memory_live_window")
        .delete()
        .eq("session_id", sessionId)

    console.info(`Memory persisted successfully for user ${userId}`)

} catch (ex) {
    // Fix 4: DB write failed – queue for retry
    console.error(`DB write failed during persistMemory for user ${userId}:`, ex)
    await supabase.from("memory_retry_queue").insert({
        user_id: userId,
        session_id: sessionId,
        combined_context: combinedContext.substring(0, 5000),
        reason: "db_write_failed",
        created_at: new Date().toISOString(),
    })
}
}

```

```

serve(async (req) => {
  const {
    action,
    userId,
    sessionId,
    userMessage,
    assistantMessage,
    messageIndex,
    assertivenessLevel,
  } = await req.json()

  // -----
  // GET CONTEXT – assemble full context package
  // -----
  if (action === "get_context") {
    const [liveWindow, sessionSummary, persistentMemory, events, cachedAnalysis] =
      await Promise.all([
        getLiveWindow(sessionId),
        getSessionSummary(sessionId),
        getPersistentMemory(userId),
        getRelevantEvents(userId),
        getCachedAnalysis(userId),
      ])

    let finalSummary = sessionSummary

    // Fix 2: Summarization now billed through billing manager
    if (liveWindow.length >= TRIGGER_THRESHOLD) {
      finalSummary = await triggerSummarization(
        userId,
        sessionId,
        liveWindow,
        sessionSummary
      )
    }
  }
})

```

```

    )
}

const trimmedWindow = liveWindow.slice(-LIVE_WINDOW_SIZE)

const finalPrompt = buildFinalPrompt(
  persistentMemory,
  finalSummary,
  events,
  cachedAnalysis,
  trimmedWindow,
  userMessage,
  assertivenessLevel ?? 3
)

// Rough token estimate to warn if approaching context limit
const estimatedTokens = Math.ceil(finalPrompt.length / 4)
const contextWarning = estimatedTokens > 3500

return new Response(
  JSON.stringify({
    prompt: finalPrompt,
    estimatedTokens,
    contextWarning,
    liveWindowCount: trimmedWindow.length,
    hasPersistentMemory: !!persistentMemory,
    hasSessionSummary: !!finalSummary,
    eventCount: events.length,
  }),
  { status: 200 }
)
}

// -----
// APPEND MESSAGE – write to live window

```

```

// -----
if (action === "append_message") {
  await appendToLiveWindow(
    userId,
    sessionId,
    "user",
    userMessage,
    messageIndex
  )

  if (assistantMessage) {
    await appendToLiveWindow(
      userId,
      sessionId,
      "assistant",
      assistantMessage,
      messageIndex + 1
    )
  }

  return new Response(
    JSON.stringify({ success: true }),
    { status: 200 }
  )
}

// -----
// WRAP UP – Fix 4: full error handling
// -----
if (action === "wrap_up") {
  await wrapUpSession(userId, sessionId)
  return new Response(
    JSON.stringify({ success: true }),
    { status: 200 }
  )
}

```



```

    return new Response(
      JSON.stringify({ error: "unknown_action" }),
      { status: 400 }
    )
  })
})

```

EDGE FUNCTION 3 — MODEL ROUTER

```

// supabase/functions/model-router/index.ts

import { serve } from "https://deno.land/std@0.168.0/http/server.ts"
import { createClient } from "https://esm.sh/@supabase/supabase-js@2"

const supabase = createClient(
  Deno.env.get("SUPABASE_URL")!,
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")!
)

// -----
// MODEL ROUTING TABLE
// Gemini 2.0 Flash – REMOVED
// Gemini 2.0 Flash Live – REMOVED
// All Gemini tasks consolidated to 2.5 Flash
// -----
const FEATURE_MODEL_MAP: Record<string, string> = {
  // Mentor – DeepSeek V4 Pro
  mentor_message: "MENTOR",

  // All Gemini tasks – Gemini 2.5 Flash only
  assistant_message: "GEMINI",
  roadmap_assistant_message: "GEMINI",
  journal_assistant_message: "GEMINI",
}

```

```

image_upload: "GEMINI",
roadmap_generation: "GEMINI",
roadmap_recalibration: "GEMINI",
weekly_review: "GEMINI",
monthly_breakdown: "GEMINI",
daily_review: "GEMINI",

// Background jobs – DeepSeek V4 Flash
// Never user facing. Always returns JSON.
summarization: "DEEPSEEK",
memory_tagging: "DEEPSEEK",
journal_classification: "DEEPSEEK",
intent_detection: "DEEPSEEK",
chat_pattern_analysis: "DEEPSEEK",

// Voice output – OpenAI TTS
// Mentor text responses only
voice_output: "OPENAI_TTS",

// Voice input – OpenAI Whisper
// Mentor sessions only
// Audio discarded after transcription
voice_input: "OPENAI_WHISPER",
}

// Output token caps per feature per tier
const OUTPUT_CAPS: Record<string, Record<string, number>> = {
  mentor_message: {
    free: 300,
    builder: 600,
    operator: 1000,
    founder: 1500,
  },
  assistant_message: {
    free: 200,
    builder: 400,
  },
}

```

```
    operator: 600,  
    founder: 800,  
  },  
  roadmap_assistant_message: {  
    free: 150,  
    builder: 400,  
    operator: 600,  
    founder: 800,  
  },  
  journal_assistant_message: {  
    free: 0,  
    builder: 300,  
    operator: 500,  
    founder: 700,  
  },  
  image_upload: {  
    free: 200,  
    builder: 400,  
    operator: 600,  
    founder: 800,  
  },  
  roadmap_generation: {  
    free: 0,  
    builder: 2000,  
    operator: 3000,  
    founder: 4000,  
  },  
  roadmap_recalibration: {  
    free: 0,  
    builder: 1500,  
    operator: 2500,  
    founder: 3500,  
  },  
  weekly_review: {  
    free: 0,  
    builder: 1000,
```

```

    operator: 2000,
    founder: 3000,
  },
  monthly_breakdown: {
    free: 0,
    builder: 1500,
    operator: 2500,
    founder: 4000,
  },
  daily_review: {
    free: 200,
    builder: 500,
    operator: 800,
    founder: 1000,
  },
}

// Call Mentor – DeepSeek V4 Pro
async function callMentor(
  prompt: string,
  maxTokens: number
): Promise<{ content: string; inputTokens: number; outputTokens: number }> {
  const response = await fetch(
    `${Deno.env.get("MENTOR_BASE_URL")}/chat/completions`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${Deno.env.get("MENTOR_API_KEY")}`,
      },
      body: JSON.stringify({
        model: Deno.env.get("MENTOR_MODEL"),
        max_tokens: maxTokens,
        messages: [{ role: "user", content: prompt }],
      })
    }
  )
}

```

```

    }),
  }
)
const data = await response.json()
return {
  content: data.choices?.[0]?.message?.content ?? "",
  inputTokens: data.usage?.prompt_tokens ?? 0,
  outputTokens: data.usage?.completion_tokens ?? 0,
}
}

// Call Gemini 2.5 Flash
// Handles all Gemini tasks including those
// previously on Gemini 2.0 Flash
async function callGemini(
  prompt: string,
  maxTokens: number,
  imageBase64?: string
): Promise<{ content: string; inputTokens: number; outputTokens: number }> {
  const parts: any[] = [{ text: prompt }]

  if (imageBase64) {
    parts.push({
      inline_data: {
        mime_type: "image/jpeg",
        data: imageBase64,
      },
    })
  }

  const response = await fetch(
    `${Deno.env.get("GEMINI_BASE_URL")}/models/${Deno.env.get("GEMINI_MODEL")}:generateContent?key=${Deno.env.get("GEMINI_API_KEY")}` ,
    {

```

```

        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            contents: [{ parts }],
            generationConfig: { maxOutputTokens: maxTokens },
        }),
    },
)

const data = await response.json()
return {
    content: data.candidates?.[0]?.content?.parts?.[0]?.text
    ?? "",
    inputTokens: data.usageMetadata?.promptTokenCount ?? 0,
    outputTokens: data.usageMetadata?.candidatesTokenCount ??
0,
}
}

// Call DeepSeek V4 Flash – background jobs only
// Always returns JSON – retry logic enforced
async function callDeepSeekFlash(
    prompt: string,
    maxTokens: number = 800
): Promise<{ content: string; inputTokens: number; outputTokens: number }> {
    const response = await fetch(
        `${Deno.env.get("DEEPSEEK_BASE_URL")}/chat/completions`,
        {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
                Authorization: `Bearer ${Deno.env.get("DEEPSEEK_API_KEY")}`,
            },
            body: JSON.stringify({

```

```

        model: Deno.env.get("DEEPSEEK_MODEL"),
        max_tokens: maxTokens,
        messages: [{ role: "user", content: prompt }],
    })),
    },
)
const data = await response.json()
return {
    content: data.choices?.[0]?.message?.content ?? "",
    inputTokens: data.usage?.prompt_tokens ?? 0,
    outputTokens: data.usage?.completion_tokens ?? 0,
}
}

// Call OpenAI TTS – mentor text responses only
// Voice hardcoded to onyx for consistency
async function callTTS(
    text: string
): Promise<{ audioBase64: string }> {
    const response = await fetch(
        `${Deno.env.get("OPENAI_TTS_BASE_URL")}/audio/speech`,
        {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
                Authorization: `Bearer ${Deno.env.get("OPENAI_TTS_API_KEY")}`,
            },
            body: JSON.stringify({
                model: Deno.env.get("OPENAI_TTS_MODEL") ?? "tts-1",
                voice: "onyx", // Hardcoded – consistent across all m
                input: text,
            })),
        }
    )
}

```

```

const audioBuffer = await response.arrayBuffer()
const audioBase64 = btoa(
  String.fromCharCode(...new Uint8Array(audioBuffer))
)

return { audioBase64 }
}

// Call OpenAI Whisper – mentor sessions only
// Audio discarded immediately after transcription
// response_format text and temperature 0 enforced
async function callWhisper(
  audioBase64: string
): Promise<{ transcription: string }> {
  const audioBuffer = Uint8Array.from(
    atob(audioBase64),
    (c) => c.charCodeAt(0)
  )

  const formData = new FormData()
  formData.append(
    "file",
    new Blob([audioBuffer], { type: "audio/webm" }),
    "audio.webm"
  )
  formData.append(
    "model",
    Deno.env.get("OPENAI_WHISPER_MODEL") ?? "whisper-1"
  )
  formData.append("response_format", "text") // Enforced – no
  timestamps needed
  formData.append("temperature", "0") // Enforced – determini
  stic transcription

  const response = await fetch(

```



```

    `${Deno.env.get("OPENAI_WHISPER_BASE_URL")}/audio/transcriptions`,
    {
      method: "POST",
      headers: {
        Authorization: `Bearer ${Deno.env.get("OPENAI_WHISPER_API_KEY")}`,
      },
      body: formData,
    }
  )

  const transcription = await response.text()

  // Audio discarded here – only transcribed text returned
  // Nothing stored beyond this point
  return { transcription }
}

serve(async (req) => {
  const {
    userId,
    sessionId,
    feature,
    prompt,
    imageBase64,
    audioBase64,
    voiceOutput,
  } = await req.json()

  if (!userId || !feature) {
    return new Response(
      JSON.stringify({ error: "userId and feature are required" }),
      { status: 400 }
    )
  }

```

```

}

// Get user tier and assertiveness level
const { data: user } = await supabase
  .from("users")
  .select("tier, assertiveness_level")
  .eq("id", userId)
  .single()

if (!user) {
  return new Response(
    JSON.stringify({ error: "user_not_found" }),
    { status: 404 }
  )
}

const tier = user.tier ?? "free"
const assertivenessLevel = user.assertiveness_level ?? 3
const modelKey = FEATURE_MODEL_MAP[feature]

if (!modelKey) {
  return new Response(
    JSON.stringify({ error: "unknown_feature" }),
    { status: 400 }
  )
}

// -----
// VOICE INPUT – Whisper transcription
// Mentor sessions only – enforced
// Audio discarded after transcription
// -----
if (feature === "voice_input") {
  if (!sessionId) {
    return new Response(
      JSON.stringify({ error: "voice_input_requires_active_"

```

```

mentor_session" })),
    { status: 400 }
  )
}

const { data: session } = await supabase
  .from("sessions")
  .select("session_type, status")
  .eq("id", sessionId)
  .single()

if (
  !session ||
  session.session_type !== "mentor" ||
  session.status !== "active"
) {
  return new Response(
    JSON.stringify({
      error: "voice_input_only_permitted_in_active_mentor_sessions",
    }),
    { status: 403 }
  )
}

try {
  const { transcription } = await callWhisper(audioBase64
4)

  return new Response(
    JSON.stringify({ transcription }),
    { status: 200 }
  )
} catch (ex) {
  return new Response(
    JSON.stringify({ error: "transcription_failed", detail: String(ex) }),

```

```

        { status: 500 }
      )
    }
  }

// -----
// BACKGROUND JOBS – DeepSeek V4 Flash only
// No XP deduction for background tasks
// JSON enforced with retry logic
// -----
if (modelKey === "DEEPSEEK") {
  let result: { content: string; inputTokens: number; outputTokens: number }

  try {
    result = await callDeepSeekFlash(prompt)
  } catch (ex) {
    return new Response(
      JSON.stringify({ error: "background_model_failed", detail: String(ex) }),
      { status: 500 }
    )
  }

  // Enforce JSON output – retry once on parse failure
  let parsed
  try {
    const clean = result.content.replace(/```json|```/g, "").trim()
    parsed = JSON.parse(clean)
  } catch {
    const retry = await callDeepSeekFlash(
      prompt +
        "\n\nYour previous response was not valid JSON. Return only the JSON object with no other content."
    )
  }
}

```

```

    try {
      const clean = retry.content.replace(/``json|``/g,
    "").trim()
      parsed = JSON.parse(clean)
    } catch {
      // Both failed – return error with raw content for de
bugging
      return new Response(
        JSON.stringify({
          error: "json_parse_failed",
          raw: result.content.substring(0, 500),
        }),
        { status: 200 }
      )
    }
  }

  return new Response(
    JSON.stringify({ result: parsed }),
    { status: 200 }
  )
}

// _____
// CHECK TIER ACCESS
// _____
const maxTokens = OUTPUT_CAPS[feature]?.[tier] ?? 300

if (maxTokens === 0) {
  return new Response(
    JSON.stringify({
      error: "feature_not_available",
      reason: `${feature} is not available on the ${tier} t
ier`,
    }),
    { status: 403 }
  )
}

```

```

    )
  }

  // -----
  // PREFLIGHT – reserve XP atomically
  // Fix 1: atomic_reserve_xp RPC used
  // -----
  const preflightResponse = await fetch(
    `${Deno.env.get("SUPABASE_URL")}/functions/v1/billing-man
ager`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${Deno.env.get("SUPABASE_SERVI
CE_ROLE_KEY")}`,
      },
      body: JSON.stringify({ action: "preflight", userId, fea
ture }),
    }
  )

  const preflight = await preflightResponse.json()

  if (!preflight.allowed) {
    return new Response(
      JSON.stringify({ error: preflight.reason }),
      { status: 402 }
    )
  }

  const reservedAmount = preflight.reservedAmount
  const reservationId = preflight.reservationId

  // -----
  // GET CONTEXT – mentor requests only

```

```

// Injects assertiveness level at session level
// -----
let finalPrompt = prompt

if (feature === "mentor_message" && sessionId) {
  const contextResponse = await fetch(
    `${Deno.env.get("SUPABASE_URL")}/functions/v1/context-engine`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${Deno.env.get("SUPABASE_SERV
VICE_ROLE_KEY")}`,
      },
      body: JSON.stringify({
        action: "get_context",
        userId,
        sessionId,
        userMessage: prompt,
        assertivenessLevel,
      }),
    }
  )

  const context = await contextResponse.json()

  if (context.prompt) {
    finalPrompt = context.prompt

    // Warn if approaching context limit
    if (context.contextWarning) {
      console.warn(`Context approaching limit for user ${us
erId}: ${context.estimatedTokens} estimated tokens`)
    }
  }
}

```

```

}

// -----
// CALL THE CORRECT MODEL
// -----
let result: { content: string; inputTokens: number; outputTokens: number }

try {
  if (modelKey === "MENTOR") {
    result = await callMentor(finalPrompt, maxTokens)
  } else if (modelKey === "GEMINI") {
    result = await callGemini(finalPrompt, maxTokens, imageBase64)
  } else {
    throw new Error(`Unhandled model key: ${modelKey}`)
  }
} catch (error) {
  // Fix 3: Cancel reservation with retry on model failure
  await fetch(
    `${Deno.env.get("SUPABASE_URL")}/functions/v1/billing-manager`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")}`,
      },
      body: JSON.stringify({
        action: "cancel",
        userId,
        reservedAmount,
        reservationId,
        feature,
      }),
    },
  )
}

```



```

    }
  )

  return new Response(
    JSON.stringify({ error: "model_call_failed", detail: String(error) }),
    { status: 500 }
  )
}

const totalTokens = result.inputTokens + result.outputTokens

// -----
// FINALIZE – true-up actual XP cost
// -----
await fetch(
  `${Deno.env.get("SUPABASE_URL")}/functions/v1/billing-manager`,
  {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${Deno.env.get("SUPABASE_SERVICE_ROLE_KEY")}`,
    },
    body: JSON.stringify({
      action: "finalize",
      userId,
      feature,
      totalTokens,
      modelUsed: modelKey,
      reservedAmount,
      reservationId,
    }),
  },
)
}

```

```

)

// -----
// APPEND TO MEMORY – mentor only
// -----
if (sessionId && feature === "mentor_message") {
  const { data: windowData } = await supabase
    .from("memory_live_window")
    .select("message_index")
    .eq("session_id", sessionId)
    .order("message_index", { ascending: false })
    .limit(1)

  const nextIndex = (windowData?.[0]?.message_index ?? -1)
+ 1

  await fetch(
    `${Deno.env.get("SUPABASE_URL")}/functions/v1/context-e
ngine`,
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${Deno.env.get("SUPABASE_SER
VICE_ROLE_KEY")}` ,
      },
      body: JSON.stringify({
        action: "append_message",
        userId,
        sessionId,
        userMessage: prompt,
        assistantMessage: result.content,
        messageIndex: nextIndex,
      }),
    }
  )
}
)

```

```

}

// -----
// VOICE OUTPUT – TTS
// Only fires when voiceOutput true AND
// feature is mentor_message
// Mentor text only – nothing else
// -----
let audioBase64Response: string | null = null

if (voiceOutput === true && feature === "mentor_message") {
  try {
    const ttsResult = await callTTS(result.content)
    audioBase64Response = ttsResult.audioBase64
  } catch (ex) {
    // TTS failure is non-fatal – return text response with
    out audio
    console.error("TTS call failed:", ex)
  }
}

return new Response(
  JSON.stringify({
    content: result.content,
    model: modelKey,
    inputTokens: result.inputTokens,
    outputTokens: result.outputTokens,
    totalTokens,
    audio: audioBase64Response,
  }),
  { status: 200 }
)
})

```

THREE THINGS TO DO IN SUPABASE BEFORE DEPLOYING:

One — Add the atomic_reserve_xp RPC function. Paste this in your SQL editor:

```
CREATE OR REPLACE FUNCTION atomic_reserve_xp(  
    p_user_id UUID,  
    p_amount INTEGER  
) RETURNS BOOLEAN AS $$  
DECLARE  
    rows_affected INTEGER;  
BEGIN  
    UPDATE user_xp  
    SET reserved_xp = reserved_xp + p_amount  
    WHERE user_id = p_user_id  
    AND (earned_xp + purchased_xp + rollover_xp - reserved_xp)  
    >= p_amount;  
  
    GET DIAGNOSTICS rows_affected = ROW_COUNT;  
    RETURN rows_affected = 1;  
END;  
$$ LANGUAGE plpgsql;
```

Two — Add the release_xp_reservation RPC function:

```
CREATE OR REPLACE FUNCTION release_xp_reservation(  
    p_user_id UUID,  
    p_amount INTEGER  
) RETURNS VOID AS $$  
BEGIN  
    UPDATE user_xp  
    SET reserved_xp = GREATEST(0, reserved_xp - p_amount)  
    WHERE user_id = p_user_id;  
END;  
$$ LANGUAGE plpgsql;
```

Three — Add the memory_retry_queue table:

```
CREATE TABLE memory_retry_queue (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  session_id UUID,
  combined_context TEXT,
  reason TEXT,
  status TEXT DEFAULT 'pending',
  created_at TIMESTAMP DEFAULT now(),
  resolved_at TIMESTAMP
);

ALTER TABLE memory_retry_queue ENABLE ROW LEVEL SECURITY;
```

PROMPT AND BRIEFING:

You are a senior backend engineer implementing three production grade Supabase Edge Functions for an AI mentorship platform called Opus Devia. Before writing or modifying any code, read this entire prompt, audit the provided code for weaknesses, and confirm your understanding of the full system. Do not write a single line of code until you have listed your findings and I have confirmed them.

UNDERSTAND THE SYSTEM FIRST: This platform has six AI models each with a strictly defined role. The three Edge Functions you are implementing are the core backend engine. They must work together as a pipeline — the model router calls the billing manager before every AI request and the context engine before every mentor request. Nothing reaches an AI model without passing through both.

The six models and their environment variables are:

- MENTOR** — reads from MENTOR_API_KEY, MENTOR_BASE_URL, MENTOR_MODEL. DeepSeek V4 Pro. Handles mentor_message only.
- GEMINI** — reads from GEMINI_API_KEY, GEMINI_BASE_URL, GEMINI_MODEL. Gemini 2.5 Flash. Handles all Gemini tasks — assistant_message, roadmap_assistant_message, journal_assistant_message, image_upload, roadmap_generation, roadmap_recalibration, weekly_review, monthly_breakdown, daily_review.
- DEEPSEEK** — reads from DEEPSEEK_API_KEY, DEEPSEEK_BASE_URL, DEEPSEEK_MODEL. DeepSeek V4 Flash. Background jobs only — summarization, memory_tagging, journal_classification, intent_detection, chat_pattern_analysis. Never user facing. Always returns JSON.
- OPENAI_TTS** —

reads from OPENAI_TTS_API_KEY, OPENAI_TTS_BASE_URL, OPENAI_TTS_MODEL. TTS-1. Voice output only. Mentor text responses only. No other content ever passed to this model.OPENAI_WHISPER — reads from OPENAI_WHISPER_API_KEY, OPENAI_WHISPER_BASE_URL, OPENAI_WHISPER_MODEL. Whisper-1. Voice input only. Mentor sessions only. Audio must be discarded immediately after transcription — never stored, never logged beyond confirmation that transcription occurred.REMOVED COMPLETELY — Gemini 2.0 Flash and Gemini 2.0 Flash Live. If any reference to either exists anywhere in the codebase, remove it. All tasks previously on Gemini 2.0 Flash are now handled by Gemini 2.5 Flash.THE THREE EDGE FUNCTIONS:The full implementation code for all three functions is provided below. Your job is not to generate new code — the code is already written. Your job is to:One — audit the provided code for bugs, logical errors, missing error handling, security vulnerabilities, and inefficiencies.Two — check that every environment variable reference uses Deno.env.get and never hardcodes a value.Three — check that the XP deduction order is correct — rollover first, purchased second, earned third — and that no pool ever goes below zero.Four — check that the session_id threading is consistent. Every memory write must include the correct session_id. The context engine and model router must pass session_id consistently on every call.Five — check that DeepSeek V4 Flash JSON output has retry logic. If the first response fails JSON parsing, it must retry once with a stricter instruction before falling back.Six — check that voice input is correctly restricted to active mentor sessions only. If a voice_input request arrives without an active mentor session, it must be rejected.Seven — check that voice output only fires when the voiceOutput flag is true AND the feature is mentor_message. No other condition should trigger TTS.Eight — check that background jobs on DeepSeek skip the XP preflight entirely since they have zero XP cost.Nine — check that the billing manager cancel action fires correctly in the catch block when a model call fails, releasing the reservation before returning the error.Ten — check that the context engine's callDeepSeekFlash function is correctly wired to DEEPSEEK environment variables and not returning an empty string placeholder.CRITICAL WEAKNESSES TO FIX IF FOUND:If the model router references model names as hardcoded strings instead of environment variables, fix it.If the billing manager's finalize action references an xp variable that is out of scope from the preflight block, fix it by re-fetching user_xp inside the finalize handler.If the context engine's wrapUpSession function does not handle the case where memory_persistent

already exists for the user and tries to insert instead of update, fix it. If any function is missing a try-catch around external API calls, add one. If the Whisper call does not enforce `response_format` as text and temperature as 0, fix it. If the TTS call does not hardcode voice as onyx regardless of what the environment variable says, fix it — voice must be consistent.

DEPLOYMENT INSTRUCTIONS: After auditing and fixing, deploy all three functions as Supabase Edge Functions. The functions must be deployed in this order: One — billing-manager first. Nothing else can work without it. Two — context-engine second. It depends on billing-manager being live. Three — model-router third. It depends on both billing-manager and context-engine being live. Each function goes in its own folder under `supabase/functions/` as `index.ts`. After deployment, test each function individually using the Supabase Edge Function tester before moving to the next one. For billing-manager test the `preflight` action with a valid `userId` and feature. For context-engine test the `get_context` action with a valid `userId` and `sessionId`. For model-router test a simple `assistant_message` request. Confirm after each test that the function responded correctly and log any errors.

DO NOT: Do not modify the billing manager's deduction order. Rollover first, purchased second, earned third. This is non-negotiable. Do not add Gemini 2.0 Flash or Gemini 2.0 Flash Live anywhere. They are removed permanently. Do not store audio from Whisper calls. Transcription only, audio discarded immediately. Do not pass anything other than mentor text responses to the TTS function. Do not skip the preflight check for any user-facing feature. Background jobs on DeepSeek are the only exception. Do not hardcode API keys, model names, or base URLs. Everything reads from environment variables

Building Sequence: (ensure you follow this sequence)

Run those three SQL blocks first, then deploy billing-manager, then context-engine, then model-router in that order.